Generators for Design Space Exploration

Martin Schoeberl

Technical University of Denmark
Embedded Systems Engineering

3 November 2026

Organization

- You presented your MVP last week
- What should you improve next?
- Today: let us measure different generated designs
- After lecture: stand-up presentations:
  - 3 minutes per group, no slides
  - What did you do last week?
  - What are your parameters?

Outline

- From generators to design space exploration (DSE)
- Revisit our MaxFinder example
- Measure area and timing with LibreLane
- Estimated fmax, latency, and throughput

Why Generators for DSE?

- One program generates many circuit variations
- We can change the architecture with a parameter
- Test all generated variants
- Measure their implementation cost
- Select a variant that meets our requirements
- What does *best* mean for your project?

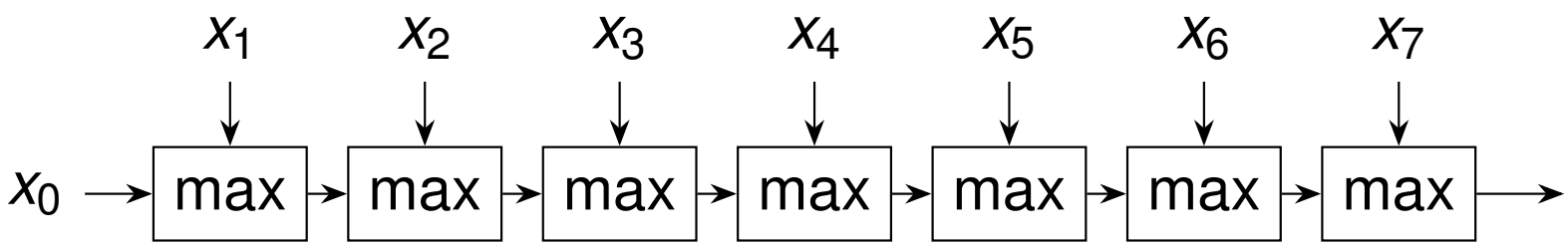
Remember MaxFinder?

- Find the maximum of a vector of unsigned values
- We used a tree reduction in lab 2
- Keep the function fixed: 8 inputs, each 16 bits
- A new input vector can arrive every cycle
- But we keep it simple without handshaking
- Can we trade area for speed?

What Do We Change?

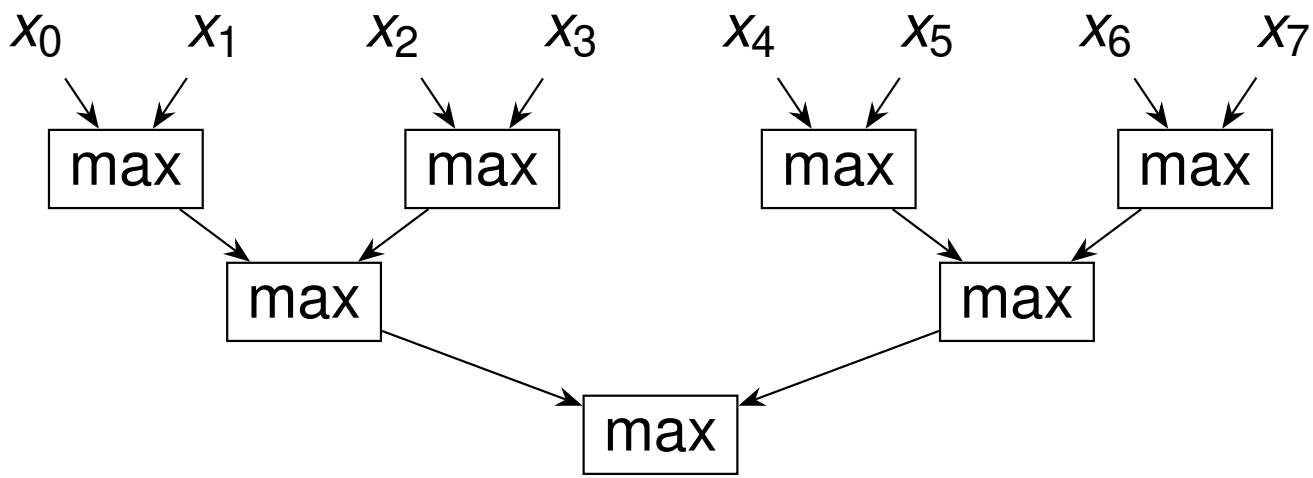
- Architecture: chain, tree, or pipelined tree
- One clock target: 20 ns (50 MHz)
- One YAML configuration for all three designs
- Keep the cell library and synthesis strategy fixed
- Compare the estimated maximum clock frequency
- Changing the bit width also changes the problem!

A Chain of Comparators



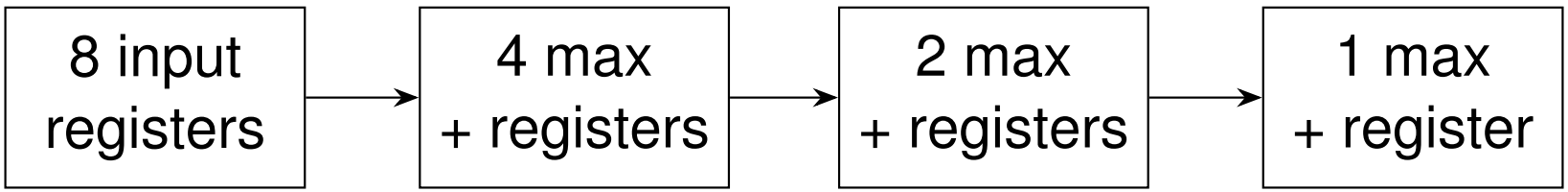
- 7 compare/select operations
- Up to 7 operations on one combinational path
- Input and output registers are not shown
- What happens when we double the input count?

A Balanced Tree



- Still 7 operations, but only 3 levels
- Input and output registers are not shown

Pipeline the Tree



- One compare/select level between registers
- More registers and more clock load
- A new vector can still arrive every cycle
- The result takes more cycles to arrive
- Is this always an improvement?

Predict Before Running the Synthesis

- Which architecture has the smallest area?
- Which one can run at the highest clock frequency?
- Which one has the lowest transaction latency?
- Synthesis may restructure our circuit
- Our RTL structure is a hypothesis, not a measurement

Generate the Chain

```
def maximum(a: UInt, b: UInt): UInt =
  Mux(a > b, a, b)

val inputs = VecInit(io.in.map(x => RegNext(x)))
io.out := RegNext(inputs.reduceLeft(maximum))
```

- Scala builds the chain during elaboration
- Input and output are registered
- A new input vector on every clock edge

Generate the Trees with reduceTree

```
// Balanced tree, with an output register
io.out := RegNext(inputs.reduceTree(maximum))

// Pipelined tree: register each compare/select
io.out := inputs.reduceTree(
  (a, b) => RegNext(maximum(a, b)))
```

- Two alternative implementations
- `inputs` is a Chisel `Vec`, not a Scala sequence
- Power-of-two input count: equal pipeline depth
- No control signals: just a data pipeline

From Vivado to LibreLane

	Vivado / FPGA	LibreLane / ASIC
Input	RTL and constraints	RTL and constraints
Target	FPGA device	Fabrication technology
Logic	LUTs, flip-flops, carry chains	Standard-cell gates and flip-flops
Full result	Bitstream	Chip layout (GDSII)

- Same questions: how much hardware, how fast?
- Different building blocks and delay models
- ASIC results do not predict FPGA LUT counts or fmax

What Is LibreLane?

- A flow that coordinates several open-source tools
- Yosys/ABC: synthesize and map RTL to standard cells
- OpenSTA: static timing analysis of the mapped netlist
- OpenROAD: placement, clock tree, and routing later
- A YAML file describes our design and constraints
- Each step writes its own netlist, logs, or reports

What Are We Mapping To?

- SKY130A: a 130 nm CMOS technology setup
- A PDK is a *process design kit*
  - Technology files and rules for implementing a chip
- Our standard-cell library: `sky130_fd_sc_hd`
  - Gates, multiplexers, flip-flops, and different drive strengths
  - Library models give each cell's area and timing behavior
- A 16-bit comparison becomes a network of these cells

How Far Do We Run?

1. Chisel elaboration: generate Verilog
2. Lint and synthesis: create a mapped cell netlist
3. Pre-placement timing analysis: estimate path delays

```
--to OpenROAD.STAPrePNR
```

- We stop here: pre-PnR means before place and route
- Later: floorplan, placement, clock tree, routing, checks
- Similar to inspecting Vivado after synthesis, before implementation
- Enough for an initial architecture comparison

One Configuration for All Three Designs

```
DESIGN_NAME: MaxFinder
VERILOG_FILES:
  - dir::MaxFinder.sv
CLOCK_PORT: clock
CLOCK_PERIOD: 20
PDK: sky130A
STD_CELL_LIBRARY: sky130_fd_sc_hd
SYNTH_STRATEGY: AREA 0
```

- The Makefile selects the generated RTL directory
- Same clock, IO assumptions, and timing corners

Let Us Run the Experiment

```
sbt test
make generate

# Inside the LibreLane environment
make chain
make tree
make pipeline
make reports
```

- Run `make generate` before the LibreLane targets
- Stop after synthesis and pre-PnR timing
- Open the reports and inspect the critical path

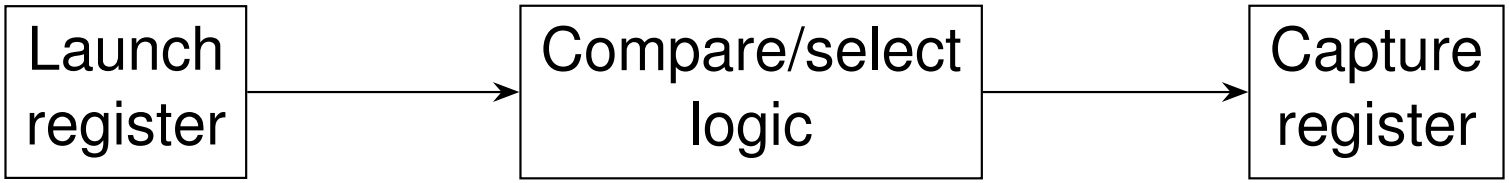
What Does the Area Estimate Count?

- Synthesis chooses cells to implement our logic
- Add the library area of every mapped cell instance

$$A_{cells} = \sum_{i} A(\mathrm{cell}_i)$$

- Includes our data registers and compare/select logic
- Larger drive strengths can cost more area
- No allowance yet for placement gaps or a clock tree
- Cell area in $\mu\mathrm{m}^2$, not a percentage of an FPGA
- The final chip needs more than these logic cells

What Does the Timing Estimate Measure?



$$T_{clock} \gtrsim t_{clk \to Q} + t_{logic} + t_{setup} + t_{uncertainty}$$

- Simplified setup check with an ideal clock
- Cell delays depend on input slew and output load
- Static timing checks paths without simulation vectors
- The limiting register path sets the clock-period estimate
- No routed-wire parasitics or implemented clock tree yet

Clock Target, Slack, and Estimated fmax

- We request 20 ns (50 MHz) for every design
- This is a constraint, not the measured result
- Setup slack = required time − arrival time
- Negative slack: that path misses the target
- The clock report estimates a minimum usable period
- Its reciprocal gives the estimated fmax
- This clock estimate does not prove IO or hold timing

Why Are There Several Timing Corners?

- Cells behave differently with process, voltage, and temperature
- Libraries describe several combinations, or *corners*
- Example: `nom_ss_100C_1v60`
  - Slow-process model, 100°C, 1.60 V
- Inspect every analyzed corner and find the lowest fmax
- Compare architectures at the same corner
- A typical-corner number alone can be misleading

Why Is This Useful for DSE?

- We measure mapped logic, not just count RTL operators
- Chain versus tree changes the depth of a path
- Pipelining adds registers and shortens each logic stage
- Hold function, library, constraints, and tools fixed
- The same early flow makes the comparison meaningful
- Physical implementation can still change the ranking
- Use this to select candidates for the more expensive flow

Latency and Throughput

- All three designs accept one vector per cycle
- At 20 ns: 50 million vectors/s, if timing passes
- Input is accepted at clock edge k
- Chain/tree output appears after edge $k + 1$
- Pipelined tree output appears after edge $k + 3$
- Acceptance-to-output latency: 20 ns versus 60 ns
- A downstream register captures one edge later

Find the Cell Area Estimate

```
# From the dse directory, after make tree
cd build/rtl/tree/runs/timing
grep 'design__instance__area' final/metrics.json
```

- Mapped standard-cell area in μm^2
- Repeat for `chain` and `pipeline`
- Also in `*-yosys-synthesis/yosys-synthesis.log`
- Search for `Chip area for module`
- Use the final top-module result, not an earlier statistic
- This is cell area, not floorplan or die area

Read the Clock Report

```
# From build/rtl/tree/runs/timing
cd *-openroad-staprepnr
less nom_ss_100C_1v60/clock.rpt
```

- Search for `report_clock_min_period`
- Find `period_min` (ns) and `fmax` (MHz)
- $f_{max} \approx 1000 / T_{min}$
- Compare the lowest fmax across all corner directories
- Use `summary.rpt` to inspect the other checks

Let Us Explore the Reports

Design	Cell area μm^2	Min. period ns	Est. fmax MHz
Chain	?	?	?
Tree	?	?	?
Pipeline	?	?	?

- Use the same corner when comparing architectures
- Which designs meet 50 MHz? What is the area cost?
- Check Yosys results for area (estimate)

Measured Area and Estimated fmax

Design	Cell area μm^2	Min. period ns	Est. fmax MHz
Chain	7729	30.66	32.62
Tree	7599	15.48	64.58
Pipeline	8996	6.11	163.69

- All runs use a 20 ns clock target
- Slow-corner clock report, before placement/routing
- LibreLane 3.0.3, SKY130A
- Which designs meet our 50 MHz target?

Which Architecture Would You Choose?

- At 50 MHz, tree and pipeline have equal throughput
- Tree: fewer cells and lower transaction latency
- Pipeline: potential for a higher clock
- About 18% more cell area for that headroom
- What changes when the throughput requirement rises?

What Have We Not Measured Yet?

- Placement, routing, and clock-tree effects
- Timing with extracted wire parasitics
- Physical checks: DRC and LVS
- Power needs a workload and switching activity
- Run full physical implementation for finalists
- Do not mix early estimates and routed results

More Architecture Parameters to Explore

- Parallelism: generate 1, 2, or 4 processing lanes
  - More hardware versus more work per cycle
- Resource sharing: one multiplier or one per operation?
  - Area versus cycles per result and control overhead
- Pipelining: where should we insert registers?
  - Clock frequency versus area and latency
- Interconnect: shared bus, crossbar, or a network?
  - Wiring and arbitration versus communication bandwidth

Explore Precision and Storage

- Arithmetic width: 8, 16, or 32 bits?
  - Area and timing versus accuracy and overflow behavior
- FIFO depth: how much buffering does the workload need?
  - Storage cost versus stalls and throughput
- Memory banking: one bank or several parallel banks?
  - Access bandwidth versus conflicts and routing
- Simulate the workload to measure accuracy or stalls
- Use suitable memory models: flops are not an SRAM macro

Apply DSE to Your Project

- AES/Ascon: round unrolling or operator sharing
- FFT: shared or parallel butterflies
- CNN/systolic array: processing elements or pipelines
- Cache: organization or lookup pipeline
- Keep precision, workload, and semantics comparable
- What is one useful parameter in your generator?

Summary

- Generators let us explore architectures
- Keep the function fixed and verify each variant
- Measure, do not just count RTL operations
- Latency and throughput are different
- Maybe make the next sprint a DSE experiment

Reading and Code

- Local example: `agile-hw/dse/`
- `README.md`: commands and assumptions
- LibreLane newcomer tutorial
- LibreLane timing closure